



TSwap Protocol Audit Report

Version 1.0

meharab.github.io

May 15, 2026

TSwap Protocol Audit Report

Meharab

May 15, 2026

Prepared by: Codereakers

Lead Auditors: - Meharab

Table of Contents

- Table of Contents
- Protocol Summary
 - TSwap Pools
 - Liquidity Providers
 - * Why would I want to add tokens to the pool?
 - * LP Example
 - Core Invariant
 - Make a swap
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings

- High
 - * [H-1] Incorrect fee calculation in `TSwapPool::getInputAmountBasedOnOutput` causes protocol to take too many tokens from users, resulting in lost fees
 - * [H-2] Lack of slippage protection in `TSwapPool::swapExactOutput` causes users to potentially receive way fewer tokens
 - * [H-3] `TSwapPool::sellPoolTokens` mismatches input and output tokens causing users to receive the incorrect amount of tokens
 - * [H-4] In `TSwapPool::_swap` the extra tokens given to users after every swap-Count breaks the protocol invariant of $x * y = k$
- Medium
 - * [M-1] `TSwapPool::deposit` is missing deadline check causing transactions to complete even after the deadline
- Low
 - * [L-1] `TSwapPool::LiquidityAdded` event has parameters out of order
 - * [L-2] Default value returned by `TSwapPool::swapExactInput` results in incorrect return value given
- Informational
 - * [I-1] `PoolFactory::PoolFactory__PoolDoesNotExist` is not used and should be removed
 - * [I-2] `PoolFactory::constructor` Lacking zero address checks
 - * [I-3] `TSwapPool::constructor` Lacking zero address check - `wethToken` & `poolToken`
 - * [I-4] `PoolFactory::createPool` should use `.symbol()` instead of `.name()`
 - * [I-5] Event is missing indexed fields
 - * [I-6] Poor test coverage
 - * [I-7] Provide better error for `TSwapPool::getInputAmountBasedOnOutput`

Protocol Summary

This project is meant to be a permissionless way for users to swap assets between each other at a fair price. You can think of T-Swap as a decentralized asset/token exchange (DEX). T-Swap is known as an Automated Market Maker (AMM) because it doesn't use a normal "order book" style exchange, instead it uses "Pools" of an asset. It is similar to Uniswap. To understand Uniswap, please watch this video: [Uniswap Explained](#)

TSwap Pools

The protocol starts as simply a `PoolFactory` contract. This contract is used to create new “pools” of tokens. It helps make sure every pool token uses the correct logic. But all the magic is in each `TSwapPool` contract.

You can think of each `TSwapPool` contract as it’s own exchange between exactly 2 assets. Any ERC20 and the WETH token. These pools allow users to permissionlessly swap between an ERC20 that has a pool and WETH. Once enough pools are created, users can easily “hop” between supported ERC20s.

For example: 1. User A has 10 USDC 2. They want to use it to buy DAI 3. They swap their 10 USDC -> WETH in the USDC/WETH pool 4. Then they swap their WETH -> DAI in the DAI/WETH pool

Every pool is a pair of TOKEN X & WETH.

There are 2 functions users can call to swap tokens in the pool. - `swapExactInput` - `swapExactOutput`

Liquidity Providers

In order for the system to work, users have to provide liquidity, aka, “add tokens into the pool”.

Why would I want to add tokens to the pool?

The TSwap protocol accrues fees from users who make swaps. Every swap has a 0.3 fee, represented in `getInputAmountBasedOnOutput` and `getOutputAmountBasedOnInput`. Each applies a 997 out of 1000 multiplier. That fee stays in the protocol.

When you deposit tokens into the protocol, you are rewarded with an LP token. You’ll notice `TSwapPool` inherits the ERC20 contract. This is because the `TSwapPool` gives out an ERC20 when Liquidity Providers (LP)s deposit tokens. This represents their share of the pool, how much they put in. When users swap funds, 0.03% of the swap stays in the pool, netting LPs a small profit.

LP Example

1. LP A adds 1,000 WETH & 1,000 USDC to the USDC/WETH pool
 1. They gain 1,000 LP tokens
2. LP B adds 500 WETH & 500 USDC to the USDC/WETH pool
 1. They gain 500 LP tokens

3. There are now 1,500 WETH & 1,500 USDC in the pool
4. User A swaps 100 USDC -> 100 WETH.
 1. The pool takes 0.3%, aka 0.3 USDC.
 2. The pool balance is now 1,400.3 WETH & 1,600 USDC
 3. aka: They send the pool 100 USDC, and the pool sends them 99.7 WETH

Note, in practice, the pool would have slightly different values than 1,400.3 WETH & 1,600 USDC due to the math below.

Core Invariant

This system works because the ratio of Token A & WETH will always stay the same. Well, for the most part. Since we add fees, our invariant technically increases.

$x * y = k$ - x = Token Balance X - y = Token Balance Y - k = The constant ratio between X & Y

y = Token Balance Y

x = Token Balance X

$x * y = k$

$x * y = (x + \Delta x) * (y - \Delta y)$

Δx = Change **of** token balance X

Δy = Change **of** token balance Y

$\beta = (\Delta y / y)$

$\alpha = (\Delta x / x)$

Final invariant equation without fees:

$\Delta x = (\beta / (1 - \beta)) * x$

$\Delta y = (\alpha / (1 + \alpha)) * y$

Invariant **with** fees

ρ = **fee** (between 0 & 1, aka a percentage)

$\gamma = (1 - \rho)$ (pronounced gamma)

$\Delta x = (\beta / (1 - \beta)) * (1/\gamma) * x$

$\Delta y = (\alpha\gamma / (1 + \alpha\gamma)) * y$

This protocol should always follow this invariant in order to keep swapping correctly!

Make a swap

After a pool has liquidity, there are 2 functions users can call to swap tokens in the pool. - swapExactInput - swapExactOutput

A user can either choose exactly how much to input (ie: I want to use 10 USDC to get however much WETH the market says it is), or they can choose exactly how much they want to get out (ie: I want to get 10 WETH from however much USDC the market says it is).

This codebase is based loosely on Uniswap v1

Disclaimer

The CodeBreakers team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

Commit Hash: 1ec3c30253423eb4199827f59cf564cc575b46db

Scope

./src/ #-> PoolFactory.sol #-> TSwapPool.sol

Roles

- Liquidity Providers: Users who have liquidity deposited into the pools. Their shares are represented by the LP ERC20 tokens. They gain a 0.3% fee every time a swap is made.
- Users: Users who want to swap tokens.

Executive Summary

Spend 48 hours and found 4 high, 1 medium, 2 low, and 7 informational in total 14 issues.

Issues found

Severity	Number of issues found
High	4
Medium	1
Low	2
Info	7
Total	14

Findings

High

[H-1] Incorrect fee calculation in TSwapPool : :getInputAmountBasedOnOutput causes protocol to take too many tokens from users, resulting in lost fees

Description: The `getInputAmountBasedOnOutput` function is intended to calculate the amount of tokens a user should deposit given an amount of tokens of output tokens. However, the function currently miscalculates the resulting amount. When calculating the fee, it scales the amount by 10_000 instead of 1_000.

Impact: Protocol takes more fees than expected from users.

Recommended Mitigation:

```
function getInputAmountBasedOnOutput(
    uint256 outputAmount,
    uint256 inputReserves,
    uint256 outputReserves
)
public
pure
revertIfZero(outputAmount)
revertIfZero(outputReserves)
returns (uint256 inputAmount)
{
-     return ((inputReserves * outputAmount) * 10_000) / ((outputReserves
↪ - outputAmount) * 997);
+     return ((inputReserves * outputAmount) * 1_000) / ((outputReserves
↪ - outputAmount) * 997);
}
```

[H-2] Lack of slippage protection in TSwapPool::swapExactOutput causes users to potentially receive way fewer tokens

Description: The swapExactOutput function does not include any sort of slippage protection. This function is similar to what is done in TSwapPool::swapExactInput, where the function specifies a minOutputAmount, the swapExactOutput function should specify a maxInputAmount.

Impact: If market conditions change before the transaction processes, the user could get a much worse swap.

Proof of Concept: 1. The price of 1 WETH right now is 1,000 USDC 2. User inputs a swapExactOutput looking for 1 WETH 1. inputToken = USDC 2. outputToken = WETH 3. outputAmount = 1 4. deadline = whatever 3. The function does not offer a maxInput amount 4. As the transaction is pending in the mempool, the market changes! And the price moves HUGE -> 1 WETH is now 10,000 USDC. 10x more than the user expected 5. The transaction completes, but the user sent the protocol 10,000 USDC instead of the expected 1,000 USDC

Recommended Mitigation: We should include a maxInputAmount so the user only has to spend up to a specific amount, and can predict how much they will spend on the protocol.

```
function swapExactOutput(
    IERC20 inputToken,
+     uint256 maxInputAmount,
    .
```



```
•
•
    inputAmount = getInputAmountBasedOnOutput(outputAmount,
↪   inputReserves, outputReserves);
+   if(inputAmount > maxInputAmount){
+       revert();
+   }
    _swap(inputToken, inputAmount, outputToken, outputAmount);
```

[H-3] TSwapPool::sellPoolTokens mismatches input and output tokens causing users to receive the incorrect amount of tokens

Description: The sellPoolTokens function is intended to allow users to easily sell pool tokens and receive WETH in exchange. Users indicate how many pool tokens they're willing to sell in the poolTokenAmount parameter. However, the function currently miscalculates the swapped amount.

This is due to the fact that the swapExactOutput function is called, whereas the swapExactInput function is the one that should be called. Because users specify the exact amount of input tokens, not output.

Impact: Users will swap the wrong amount of tokens, which is a severe disruption of protocol functionality.

Recommended Mitigation:

Consider changing the implementation to use swapExactInput instead of swapExactOutput. Note that this would also require changing the sellPoolTokens function to accept a new parameter (ie minWethToReceive to be passed to swapExactInput)

```
function sellPoolTokens(
    uint256 poolTokenAmount,
+   uint256 minWethToReceive,
) external returns (uint256 wethAmount) {
-   return swapExactOutput(i_poolToken, i_wethToken, poolTokenAmount,
↪   uint64(block.timestamp));
+   return swapExactInput(i_poolToken, poolTokenAmount, i_wethToken,
↪   minWethToReceive, uint64(block.timestamp));
}
```

Additionally, it might be wise to add a deadline to the function, as there is currently no deadline.

[H-4] In TSwapPool : : _swap the extra tokens given to users after every swapCount breaks the protocol invariant of $x * y = k$

Description: The protocol follows a strict invariant of $x * y = k$. Where: - x: The balance of the pool token - y: The balance of WETH - k: The constant product of the two balances

This means, that whenever the balances change in the protocol, the ratio between the two amounts should remain constant, hence the k. However, this is broken due to the extra incentive in the _swap function. Meaning that over time the protocol funds will be drained.

The follow block of code is responsible for the issue.

```
swap_count++;  
if (swap_count >= SWAP_COUNT_MAX) {  
    swap_count = 0;  
    outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000_000);  
}
```

Impact: A user could maliciously drain the protocol of funds by doing a lot of swaps and collecting the extra incentive given out by the protocol.

Most simply put, the protocol's core invariant is broken.

Proof of Concept: 1. A user swaps 10 times, and collects the extra incentive of 1_000_000_000_000_000_000 tokens 2. That user continues to swap untill all the protocol funds are drained

Proof Of Code

Place the following into TSwapPool.t.sol.

```
function testInvariantBroken() public {  
    vm.startPrank(liquidityProvider);  
    weth.approve(address(pool), 100e18);  
    poolToken.approve(address(pool), 100e18);  
    pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));  
    vm.stopPrank();  
  
    uint256 outputWeth = 1e17;  
  
    vm.startPrank(user);  
    poolToken.approve(address(pool), type(uint256).max);  
    poolToken.mint(user, 100e18);  
    pool.swapExactOutput(poolToken, weth, outputWeth,  
        uint64(block.timestamp));  
}
```

```
        pool.swapExactOutput(poolToken, weth, outputWeth,
↪   uint64(block.timestamp));
        pool.swapExactOutput(poolToken, weth, outputWeth,
↪   uint64(block.timestamp));
        pool.swapExactOutput(poolToken, weth, outputWeth,
↪   uint64(block.timestamp));
        pool.swapExactOutput(poolToken, weth, outputWeth,
↪   uint64(block.timestamp));
        pool.swapExactOutput(poolToken, weth, outputWeth,
↪   uint64(block.timestamp));
        pool.swapExactOutput(poolToken, weth, outputWeth,
↪   uint64(block.timestamp));
        pool.swapExactOutput(poolToken, weth, outputWeth,
↪   uint64(block.timestamp));
        pool.swapExactOutput(poolToken, weth, outputWeth,
↪   uint64(block.timestamp));
        pool.swapExactOutput(poolToken, weth, outputWeth,
↪   uint64(block.timestamp));

        int256 startingY = int256(weth.balanceOf(address(pool)));
        int256 expectedDeltaY = int256(-1) * int256(outputWeth);

        pool.swapExactOutput(poolToken, weth, outputWeth,
↪   uint64(block.timestamp));
        vm.stopPrank();

        uint256 endingY = weth.balanceOf(address(pool));
        int256 actualDeltaY = int256(endingY) - int256(startingY);
        assertEq(actualDeltaY, expectedDeltaY);
    }
```

Recommended Mitigation: Remove the extra incentive mechanism. If you want to keep this in, we should account for the change in the $x * y = k$ protocol invariant. Or, we should set aside tokens in the same way we do with fees.

```
-         swap_count++;
-         // Fee-on-transfer
-         if (swap_count >= SWAP_COUNT_MAX) {
-             swap_count = 0;
-             outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000_000);
-         }
```

Medium

[M-1] TSwapPool::deposit is missing deadline check causing transactions to complete even after the deadline

Description: The `deposit` function accepts a deadline parameter, which according to the documentation is “The deadline for the transaction to be completed by”. However, this parameter is never used. As a consequence, operations that add liquidity to the pool might be executed at unexpected times, in market conditions where the deposit rate is unfavorable.

Impact: Transactions could be sent when market conditions are unfavorable to deposit, even when adding a deadline parameter.

Proof of Concept: The `deadline` parameter is unused.

Warning (5667): Unused **function** parameter. **Remove** or comment out the variable name to silence **this** warning.

```
--> src/TSwapPool.sol:96:9:
    |
96 |         uint64 deadline
    |         ^^^^^^^^^^^^^^^^^
```

Recommended Mitigation: Consider making the following change to the function.

```
function deposit(
    uint256 wethToDeposit,
    uint256 minimumLiquidityTokensToMint, // LP tokens -> if empty, we
    can pick 100% (100% == 17 tokens)
    uint256 maximumPoolTokensToDeposit,
    uint64 deadline
)
    external
+   revertIfDeadlinePassed(deadline)
    revertIfZero(wethToDeposit)
    returns (uint256 liquidityTokensToMint)
{
```

Low

[L-1] TSwapPool::LiquidityAdded event has parameters out of order

Description: When the `LiquidityAdded` event is emitted in the `TSwapPool::_addLiquidityMintAndTransfer` function, it logs values in an incorrect order. The `poolTokensToDeposit` value should go in the

third parameter position, whereas the `wethToDeposit` value should go second.

Impact: Event emission is incorrect, leading to off-chain functions potentially malfunctioning.

Recommended Mitigation:

```
- emit LiquidityAdded(msg.sender, poolTokensToDeposit, wethToDeposit);  
+ emit LiquidityAdded(msg.sender, wethToDeposit, poolTokensToDeposit);
```

[L-2] Default value returned by TSwapPool : : swapExactInput results in incorrect return value given

Description: The `swapExactInput` function is expected to return the actual amount of tokens bought by the caller. However, while it declares the named return value `output` it is never assigned a value, nor uses an explicit return statement.

Impact: The return value will always be 0, giving incorrect information to the caller.

Recommended Mitigation:

```
    {  
        uint256 inputReserves = inputToken.balanceOf(address(this));  
        uint256 outputReserves = outputToken.balanceOf(address(this));  
  
-        uint256 outputAmount = getOutputAmountBasedOnInput(inputAmount,  
↪ inputReserves, outputReserves);  
+        output = getOutputAmountBasedOnInput(inputAmount, inputReserves,  
↪ outputReserves);  
  
-        if (output < minOutputAmount) {  
-            revert TSwapPool__OutputTooLow(outputAmount, minOutputAmount);  
+        if (output < minOutputAmount) {  
+            revert TSwapPool__OutputTooLow(outputAmount, minOutputAmount);  
        }  
  
-        _swap(inputToken, inputAmount, outputToken, outputAmount);  
+        _swap(inputToken, inputAmount, outputToken, output);  
    }
```

Informational

[I-1] PoolFactory::PoolFactory__PoolDoesNotExist is not used and should be removed

```
- error PoolFactory__PoolDoesNotExist(address tokenAddress);
```

[I-2] PoolFactory::constructor Lacking zero address checks

```
    constructor(address wethToken) {  
+       if(wethToken == address(0)) {  
+           revert();  
+       }  
        i_wethToken = wethToken;  
    }
```

[I-3] TSwapPool::constructor Lacking zero address check - wethToken & poolToken

```
constructor(  
    address poolToken,  
    address wethToken,  
    string memory liquidityTokenName,  
    string memory liquidityTokenSymbol  
)  
    ERC20(liquidityTokenName, liquidityTokenSymbol)  
{  
+   if(wethToken || poolToken == address(0)){  
+       revert();  
+   }  
    i_wethToken = IERC20(wethToken);  
    i_poolToken = IERC20(poolToken);  
}
```

[I-4] PoolFacotry::createPool should use .symbol() instead of .name()

```
-       string memory liquidityTokenSymbol = string.concat("ts",  
↪     IERC20(tokenAddress).name());  
+       string memory liquidityTokenSymbol = string.concat("ts",  
↪     IERC20(tokenAddress).symbol());
```

[I-5] Event is missing indexed fields

Index event fields make the field more quickly accessible to off-chain tools that parse events. However, note that each index field costs extra gas during emission, so it's not necessarily best to index the maximum allowed per event (three fields). Each event should use three indexed fields if there are three or more fields, and gas usage is not particularly of concern for the events in question. If there are fewer than three fields, all of the fields should be indexed.

- Found in src/TSwapPool.sol: Line: 44
- Found in src/PoolFactory.sol: Line: 37
- Found in src/TSwapPool.sol: Line: 46
- Found in src/TSwapPool.sol: Line: 43

[I-6] Poor test coverage

Running tests...

File	% Lines	% Statements	% Branches	% Funcs
src/PoolFactory.sol	100.00% (11/11)	100.00% (16/16)	100.00% (2/2)	100.00%
src/TSwapPool.sol	54.84% (34/62)	59.14% (55/93)	33.33% (6/18)	37.50% (6/16)

Recommended Mitigation: Aim to get test coverage up to over 90% for all files.

[I-7] Provide better error for TSwapPool::getInputAmountBasedOnOutput

When outputReserves & outputAmount are the same, TSwapPool::getInputAmountBasedOnOutput will revert with arithmetic divide by zero. This is not very helpful for users. Consider adding a custom error message.

```
+     error TSwapPool__CannotRemoveEntireBalance();
+
+
+     if (outputReserves, outputAmount) {revert
+ ↪     TSwapPool__CannotRemoveEntireBalance();}
+         return (inputReserves * outputAmount) / ((outputReserves -
+ ↪     outputAmount));
```